



UNIVERSITÀ DEGLI STUDI DI PADOVA

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica

Insegnamento di Ingegneria del Software

Anno Accademico 2026/2027

Valutazione Capitolati

Sintesi comparativa e motivazione della scelta

Gruppo	Bit Crusher
Email	bitcrusherteam@gmail.com
Versione	1.1.0
Data Redazione	04 Novembre 2026
Responsabile Redazione	Marco Bolletin
Responsabile di Progetto	Alex Singh
Uso	Esterno

Registro delle Modifiche

Vers.	Data	Autore	Verificatore	Descrizione
1.1.0	04/11/2026	Alex Singh	-	Approvazione per il rilascio del documento per la candidatura del C5.
1.0.2	03/11/2026	Marco Bollettin	Pietro L. Acampora	Sostituzione della scelta capitolato con l'estensione delle conclusioni e motivazioni scelta.
1.0.1	03/11/2026	Marco Bollettin	Antonio Tufano	Espansione della sottosezione relativa al capitolato 5.
1.0.0	28/10/2026	Riccardo Pozzato	-	Approvazione per il rilascio del documento per il capitolato C6.
0.1.5	28/10/2026	Alex Singh	Pietro L. Acampora	Completamento della sottosezione 4.1.4.
0.1.4	28/10/2026	Alex Singh	Antonio Tufano	Aggiunta della sezione per la scelta del capitolato.
0.1.3	28/10/2026	Alex Singh	Antonio Tufano	Espansione della sezione delle conclusioni e scelta finale.
0.1.2	28/10/2026	Alex Singh	Alexandru Mitu	Fusione del documento di studio della fattibilità con la valutazione dei capitolati.
0.1.1	27/10/2026	Antonio Tufano / Pietro L. Acampora	Alex Singh	Prima stesura e inserimento analisi dei capitolati.
0.1.0	26/10/2026	Antonio Tufano / Pietro L. Acampora	Alex Singh	Impostazione dell'ossatura iniziale del documento.

Indice

Registro delle Modifiche **1**

1 Introduzione **3**
1.1 Scopo del Documento 3
1.2 Criteri di Valutazione 3

2 Sintesi delle Valutazioni **3**

3 Analisi Dettagliata per Capitolato **4**
3.1 C0: AIRA: AI Requests Automation 4
3.2 C1: AURA: AI for Understanding, Reasoning and Automation 4
3.2.1 Descrizione generale 4
3.2.2 Obiettivi e finalità 4
3.2.3 Tecnologie richieste 4
3.2.4 Vincoli e requisiti 5
3.2.5 Aspetti positivi 5
3.2.6 Criticità e rischi 5
3.3 C2: NEXUM Concierge 5
3.4 C3: Fossdot: Smart Speaker Open Source estensibile e rispettoso della privacy . 6
3.4.1 Descrizione generale 6
3.4.2 Obiettivi e finalità 6
3.4.3 Tecnologie richieste 6
3.4.4 Vincoli e requisiti 6
3.4.5 Aspetti positivi 7
3.4.6 Criticità e rischi 7
3.5 C4: Gestione Veicoli e Servizi Post-Vendita 7
3.6 C5: RareCare: Piattaforma AI per la raccolta dati e il supporto decisionale nelle malattie rare 8
3.6.1 Descrizione generale 8
3.6.2 Obiettivi e finalità 8
3.6.3 Tecnologie richieste 8
3.6.4 Vincoli e requisiti 8
3.6.5 Aspetti positivi 9
3.6.6 Criticità e rischi 9
3.7 C6: Sistema AI di caching semantico e dashboard FinOps (Var Group) 9
3.7.1 Descrizione generale 9
3.7.2 Obiettivi e finalità 9
3.7.3 Tecnologie richieste 10
3.7.4 Vincoli e requisiti 10
3.7.5 Aspetti positivi 10
3.7.6 Criticità e rischi 10
3.8 C7: SmartDashboard 11
3.9 C8: Sonar: Documentazione vivente di uno store Shopify 11
3.10 C9: SpecCheck: Skills OpenCode per tracciabilità tra casi d'uso, requisiti e codice sorgente 12

4 Conclusioni e Scelta Finale **13**

1 Introduzione

1.1 Scopo del Documento

Il presente documento espone il processo di valutazione effettuato dal gruppo **Bit Crusher** in merito ai capitolati d'appalto proposti per il Progetto Didattico. L'analisi ha portato alla selezione del progetto più in linea con le aspirazioni formative, le competenze pregresse e gli obiettivi di crescita del team.

1.2 Criteri di Valutazione

Per garantire un approccio oggettivo e sistematico, ogni proposta è stata valutata assegnando un punteggio da 1 (minimo) a 5 (massimo) sui seguenti quattro assi:

- **V1 - Interesse nel Dominio:** Attrattiva verso l'ambito applicativo e le tematiche del progetto.
- **V2 - Interesse Tecnologico:** Stimolo offerto dallo stack tecnologico e dalle architetture (Cloud, AI, ecc.).
- **V3 - Fattibilità e Complessità:** Bilanciamento tra la sfida proposta, le competenze attuali e i vincoli temporali.
- **V4 - Chiarezza e Supporto:** Qualità della documentazione iniziale e disponibilità dell'azienda durante gli incontri.

2 Sintesi delle Valutazioni

La tabella seguente riassume i punteggi assegnati dal gruppo a valle degli incontri con i referenti aziendali e delle riunioni interne.

Capitolato	V1	V2	V3	V4	Totale
~~**C0:** AIRA: AI Requests Automation~~	3	3	3	3	**12**
~~**C1:** AURA: AI for Understanding...~~	4	3	4	3	**14**
~~**C2:** Concierge~~	2	3	3	4	**12**
~~**C3:** Fossdot: Smart Speaker Open Source...~~	5	4	3	4	**16**
~~**C4:** Gestione Veicoli e Servizi Post-Vendita~~	2	2	4	3	**11**
C5: RareCare: Piattaforma AI...	4	3	3	4	**14**
~~**C6:** Sistema AI di caching semantico e dashboard FinOps~~	5	5	4	5	**19**
C7: Smart Dashboard: Piattaforma di BI...	3	4	3	3	**13**
C8: Sonar: Documentazione vivente...	3	3	4	3	**13**
C9: SpecCheck: Skills OpenCode...	2	3	3	3	**11**

Legenda: V1 = Dominio, V2 = Tecnologie, V3 = Complessità, V4 = Supporto. Il punteggio massimo ottenibile è 20.

3 Analisi Dettagliata per Capitolato

3.1 C0: AIRA: AI Requests Automation

Descrizione: Il progetto mira a realizzare un tool multimodale basato su Intelligenza Artificiale che gestisce le richieste B2B arrivate in forma non strutturata (e-mail con allegati, note vocali, immagini, messaggi). Il sistema monitora in modo continuo i canali configurati, applica un filtro di sicurezza per l'isolamento dei dati e utilizza speech-to-text, OCR e un LLM per estrarre entità e intenzioni, convertendole in dati strutturati. In base all'entità rilevata, avvia automaticamente azioni configurabili, come chiamate API REST verso B2B, ERP, CRM e ticketing o l'invio di e-mail di notifica, eliminando i passaggi manuali. Un'interfaccia amministrativa web consente di gestire i canali, consultare i log e configurare la persistenza dei dati.
Tecnologie: Java, Spring Boot, Angular, TypeScript, LangChain4j, LLM, Whisper, Google Speech, Tesseract, EasyOCR, SQLite/MariaDB/PostgreSQL, API REST, JSON/YAML

- **Pro:** Casi d'uso B2B reali; utilità aziendale immediata; integrazione IA pratica.
- **Contro:** Stack tecnologico imposto; scarsa libertà decisionale sul backend; logiche aziendali preesistenti rigide.

Sintesi del motivo di scarto: Impone uno stack legacy e logiche aziendali chiuse, negando la libertà di progettare un'architettura backend pulita e moderna in autonomia.

3.2 C1: AURA: AI for Understanding, Reasoning and Automation

3.2.1 Descrizione generale. AURA, proposto da Vimar S.p.A., chiede di collegare i servizi commerciali di IA generativa (Claude, ChatGPT, Copilot, Gemini) agli impianti domotici Vimar View Wireless, per interrogarli e comandarli in linguaggio naturale. L'impianto è una rete Bluetooth Mesh di dispositivi da incasso collegata, tramite gateway, al cloud Vimar, che espone la KNX IoT 3rd Party API: un'interfaccia REST standard in cui ogni dispositivo è un insieme di *funzionalità* con i propri *datapoint*, collocate in una *location*. Le macro-componenti sono tre: un'infrastruttura cloud-ready, un MCP server che fa da adattatore verso KNX IoT (con il suo client) e l'agente IA esterno con cui l'utente conversa. Vimar fornisce a ogni gruppo un kit fisico (luci, tapparelle, termostato, sensori UWB), l'accesso alle API e, se serve, spazio su AWS.

3.2.2 Obiettivi e finalità. La finalità è un dimostratore IoT in concomitanza all'AI che mostri fattibilità e potenzialità delle API Vimar: invece dell'ennesimo assistente dedicato, ognuno usa il servizio IA che preferisce attraverso standard aperti, in linea con la strategia aziendale di valorizzare le integrazioni di terze parti. Non si vuole sostituire l'app esistente, ma aprire un canale complementare e una base per evoluzioni future (l'app mobile mostrata è fuori scope però). In concreto: le tre macro-componenti funzionanti sui 17 casi d'uso base e sugli 11 avanzati obbligatori (ambiguità, sinonimi, richieste implicite, conferme, errori), più architettura C4, un'analisi sullo sviluppo basata sulle specifiche con stima dei costi AWS, documentazione MCP come sito statico, manuale d'installazione, release pubblica, demo e report dei test.

3.2.3 Tecnologie richieste. Vimar lascia ampio margine, ma i paletti sono netti: Docker con docker-compose per un IaC replicabile con un solo comando e un `config.env` documentato, compatibilità di rilascio su AWS, MCP per server e client, KNX IoT 3rd Party API (v2.1.0) con OAuth2, notifiche push via *Subscription* (niente polling, salvo accordi), un servizio IA commerciale e Git pubblico con licenza open source, occhio alle licenze delle dipendenze. Tra i suggerimenti troviamo Python con FastMCP (o TypeScript/Java, stesso SDK per client e server;

quale FastMCP intendano va chiarito, nella v2 dell'SDK ufficiale non c'è più), AWS Lightsail o EC2, la CI con GitHub Runners e Copilot con spec-kit.

3.2.4 Vincoli e requisiti. Conviene dividerli in obbligatori, opzionali e nice-to-have. Gli obbligatori seguono le macro-componenti: MCP server con gli endpoint KNX IoT Authorization, Datapoint, Funzionalità, Information, Location, Subscription e Timeseries, usabile da client di terze parti, con lettura/scrittura sull'impianto, ricostruzione delle relazioni tra entità e OAuth2; client MCP che parla solo con il nostro server; modello IA esterno con login MyVimar e impianto selezionato mantenuto lungo la conversazione. Poi i test: unità e integrazione con coverage minima del 75%, end-to-end almeno all'80% sui requisiti. Opzionali il deploy effettivo su AWS (anche serverless + container), l'endpoint Device, il client in VSCode o via CLI/TUI e Terraform, CDK o Ansible accanto al compose. Infine i nice-to-have: multi-impianto, azioni condizionate («se ci sono luci accese, spegnile»), scenari composti, storico con relative operazioni, trigger ed eventi.

3.2.5 Aspetti positivi. Dal confronto col gruppo emerge che AURA è relativamente complesso, e l'abbiamo concordato come un aspetto positivo: richiede di lavorare con standard e protocolli veri (MCP, KNX IoT, OAuth2), utili alla nostra crescita, partendo da ciò che oggi sta dilagando per portare la smart home allo stadio successivo. Ci ha colpito poi che Vimar non si affidi ad un'unità centralizzata come il classico Home Assistant, ma esponga un endpoint verso servizi di terzi (Anthropic, OpenAI, Google, Microsoft), con una separazione netta delle responsabilità tra le parti (nessuno vieta di affiancarci un Home Assistant).

3.2.6 Criticità e rischi. MCP corre veloce, e la cosa più spaventosa sono proprio i breaking change: la revisione del 28 luglio 2026 ne ha portati di pesanti (spariscono sessioni e handshake, la registrazione dinamica dei client è deprecata) e, nonostante la nuova policy di deprecazione a dodici mesi, i servizi commerciali possono parlare versioni diverse del protocollo: la compatibilità è a carico nostro. Il passaggio allo stateless risolve il conflitto con il load balancer ma sposta il problema: l'impianto selezionato non vive più nella sessione e va gestito con handle espliciti. L'autenticazione resta la criticità più grossa: due layer di token (client → MCP, MCP → KNX IoT) tra refresh, revoca e scope, niente di out-of-the-box. Il problema nascosto, emerso alla seconda riunione, è la risoluzione delle ambiguità: non è un limite dell'MCP ma di prompt engineering (una mitigazione l'abbiamo pensata, ma non è banale). Poi il compose che si fa fragile al crescere dell'infrastruttura, AWS (opzionale, attenzione) come collo di bottiglia di networking, prompt injection e token exposure (patata bollente con un repo pubblico) e la dipendenza da Vimar, la cui eventuale v2.2 dell'API dovrà essere retro-compatibile in caso.

Sintesi del motivo di scarto: Candidatura scartata in quanto non sufficientemente stimolante rispetto ad altre scelte.

3.3 C2: NEXUM Concierge

Descrizione: Il progetto Concierge, proposto da Eggon, mira a realizzare un assistente virtuale conversazionale per la piattaforma di servizi HR NEXUM. Il tool consentirà ai dipendenti di accedere a informazioni e completare operazioni, come la richiesta di ferie e permessi, direttamente via chat. Il sistema sfrutta un modello linguistico per mantenere il contesto, consultare la knowledge base documentale (regolamenti e manuali) e interfacciarsi con i servizi aziendali per la lettura dei dati e l'invio delle richieste. Include inoltre un backoffice per gestire i documenti aziendali.
Tecnologie: Angular, TypeScript, PrimeNG, Tailwind CSS, Ruby on Rails, PostgreSQL, Redis, Sidekiq, LLM/stack AI (da concordare), OAuth2, JWT

- **Pro:** Mentorship e supporto costanti; architettura di base già definita; forte focus sulla User Experience.
- **Contro:** Interazione forzata con backend chiusi; dipendenza totale da API esterne; sviluppo sbilanciato sulle logiche frontend.

Sintesi del motivo di scarto: L'obbligo di appoggiarsi a un backend "black-box" aziendale preesistente azzera totalmente la possibilità di strutturare il database e le API da zero.

3.4 C3: Fossdot: Smart Speaker Open Source estensibile e rispettoso della privacy

3.4.1 Descrizione generale. FossDot, proposto da Uranion EMEA Ltd, chiede uno smart speaker open source, funzionalmente analogo ad un Amazon Echo Dot, coprendo l'intero stack dall'hardware ai servizi. Alla base c'è l'hardware di riferimento (SBC o microcontrollore, array di microfoni, altoparlante, amplificatore e feedback luminoso), da racchiudere in un contenitore da prodotto finale; sopra, il firmware acquisisce l'audio, rileva la wake word (o il push-to-talk), invia lo stream al server e riproduce la risposta. Dopodiché il servizio di bridging implementa la pipeline STT $\rightarrow$ LLM $\rightarrow$ TTS con un'astrazione che rende il provider IA (cloud o locale) sostituibile, più un client MCP per servizi esterni, Skills e Agents; infine un'interfaccia web registra i dispositivi, li associa a un account e ne gestisce provider e interconnessione.

3.4.2 Obiettivi e finalità. Ciò a cui mira è un'alternativa aperta agli smart speaker commerciali, chiusi e legati al cloud del produttore e ad un unico assistente: il provider IA si sceglie e si sostituisce liberamente, il sistema è self-hosted e ispezionabile, può elaborare la voce tutta in locale (utile anche dove il GDPR è stringente) ed è ricostruibile da chiunque, per cui non muore quando qualcuno spegne un cloud. L'MVP chiede hardware validato con distinta base e prototipo, firmware con wake word o push-to-talk, bridging verso almeno un provider, registrazione di un dispositivo e flusso end-to-end su dispositivo fisico. La consegna completa aggiunge provider commutabili (es. Ollama), MCP con uno o due servizi esterni, Skills e Agents a voce, multi-dispositivo (stanze, annunci, riproduzione sincronizzata, intercom), firmware in Rust e/o OTA e hardening (segreti, autenticazione, STT/TTS locali).

3.4.3 Tecnologie richieste. Lo stack non è imposto: tocca al gruppo scegliere e motivare rispetto al caso d'uso. Fatto sta che dei punti fermi ci sono: rilascio dell'intero progetto (software e sorgenti hardware) con licenza permissiva, Apache 2.0 o MIT, MCP lato bridging e un'applicazione web per la gestione; il firmware in Rust è preferenziale (un valore aggiunto, non un vincolo) e sono incoraggiate containerizzazione e automazione di build e rilascio. I riferimenti, non prescrittivi, sono Rust embedded con Embassy, Raspberry Pi, Seeed ReSpeaker, openWakeWord, Silero VAD, faster-whisper, Piper, Ollama e Axum/Tokio, con Willow e Rhasspy/Home Assistant come prior art.

3.4.4 Vincoli e requisiti. La licenza è un punto di pressione: permissiva, con la proprietà che resta al gruppo e un repository pubblico. La validazione va fatta su hardware reale (wake word $\rightarrow$ domanda $\rightarrow$ risposta su dispositivo fisico), la simulazione è ammessa solo nei test. La proponente non fornisce dataset e i flussi vocali sono dati personali, quindi il trattamento va progettato secondo il GDPR. I test vanno concordati per ogni componente, con coverage di riferimento all'80% e report per bridging e interfaccia, mentre per il firmware (situazione più delicata) si decide caso per caso; le scelte vanno documentate con motivazioni e problemi aperti. Restano impliciti, con priorità da concordare, il canale dispositivo-server e la scoperta in rete.

3.4.5 Aspetti positivi. Il perimetro è modulare: la scelta tra MVP e consegna completa spetta a noi, quindi preventivo e rischio si dimensionano sulle nostre capacità reali. La libertà tecnologica non è un salto nel vuoto, perché il capitolato indica librerie e prior art concreti, e l'architettura si scompone bene, sia per lavorare in parallelo in sette sia per pattern riconoscibili come Strategy e Adapter sui provider. I criteri di accettazione sono verificabili, non servono dati né accessi della proponente, la pipeline può girare tutta in locale e la fattibilità è già dimostrata da dispositivi open come Home Assistant Voice PE (59 €). Da non sottovalutare il supporto, tra formazione, SAL e tech dive con i loro software architect, e il valore di portfolio di un prodotto fisico tutto nostro.

3.4.6 Criticità e rischi. Il primo nodo è l'architettura del dispositivo: i riferimenti puntano a due strade incompatibili (microcontrollore con Rust `no_std` ed Embassy, oppure SBC Linux con openWakeWord in Python) e la scelta decide wake word, firmware e distinta base, quindi va chiusa con una PoC prima dell'RTB. L'audio a distanza (eco, beamforming, rumore) è elaborazione del segnale, fuori dal nostro percorso: senza un DSP integrato la wake word scatta a vuoto. Rust embedded è maturo solo in parte (in esp-hal 1.0 I2S e Wi-Fi hanno API ancora instabili) e i motori di wake word su MCU sono in C/C++. Ci sono poi ambiguità (l'MVP ammette il push-to-talk ma il collaudo è «wake word → risposta»; mancano latenza, lingua e soglie di falsi positivi), licenze che stridono con Apache/MIT (modelli di openWakeWord in 4.0, Piper passato a GPLv3), hardware e costi API non attribuiti e un supporto centrato su AI e software proprio dove il rischio è embedded. Infine la privacy, con un microfono sempre in ascolto.

Sintesi del motivo di scarto: Tenuto in considerazione ma superato da proposte maggiormente in linea con i nostri interessi backend.

3.5 C4: Gestione Veicoli e Servizi Post-Vendita

Descrizione: Il progetto, proposto da M31, mira a sviluppare una piattaforma distribuita per la gestione dei veicoli e dei servizi post-vendita nel settore automotive. Il sistema connette la casa automobilistica (cataloghi, veicoli e campagne di richiamo), le concessionarie (vendite e manutenzioni) e i proprietari (area personale, consultazione dati e appuntamenti).

Tecnologie: Node.js, Nest.js, TypeScript, Go, Angular, PostgreSQL, MongoDB, Redis, NATS o Apache Kafka, Kubernete, Prometheus e Grafana

- **Pro:** Architettura a microservizi moderna; modellazione dati relazionale estesa; settore automotive in forte crescita.
- **Contro:** Dominio dati estremamente frammentato; rischio di logiche di astrazione eccessive; integrazioni di rete complesse.

Sintesi: L'astrazione spinta dei microservizi frammenta inutilmente il sistema, quando una base di dati relazionale solida e centralizzata sarebbe nettamente più efficiente.

3.6 C5: RareCare: Piattaforma AI per la raccolta dati e il supporto decisionale nelle malattie rare

3.6.1 Descrizione generale. RareCare Navigator, proposto da Miriade srl, chiede una piattaforma digitale per le famiglie che convivono con una malattia rara, a partire dalla Sindrome di Sanfilippo (mucopolisaccaridosi di tipo III), patologia neurodegenerativa progressiva. Va detto che il problema di fondo non è clinico ma organizzativo: oggi è la famiglia a fare da collante, a mano, tra centri specialistici, pediatri, terapisti, scuola, servizi sociali ed enti amministrativi. Il sistema affianca il Care Navigator (con funzioni di case manager) e fa da connettore sicuro tra gli attori della rete di cura, con quattro strumenti: un assistente RAG vincolato a fonti verificate, checklist e scadenze per fase clinica, un fascicolo digitale cifrato con permessi granulari e un modello a plugin, il Disease Pack, che separa il core da ciò che dipende dalla patologia. L'allegato propone un'architettura AWS serverless in otto contesti di dominio, presentata come ipotesi e non come vincolo.

3.6.2 Obiettivi e finalità. L'obiettivo principe è trasformare un percorso di cura frammentato in un'esperienza integrata, dove ogni attore vede soltanto ciò che gli compete; la finalità di più lungo respiro è la replicabilità: una nuova patologia dovrebbe richiedere solo un nuovo Disease Pack (configurazione e contenuti, non codice). Il percorso è incrementale: l'MVP comprende login con MFA, rete di cura con permessi, fascicolo digitale, checklist e scadenze con notifiche e un assistente RAG su un corpus ridotto, validato sul campo con l'associazione Sanfilippo Fighters; seguono l'estrazione di entità con revisione umana e le checklist generate dai PAI/PEI, mentre assistente vocale, mappatura geospaziale e analisi predittiva restano evoluzioni opzionali. Cosa non scontata, Miriade ammette che la soluzione completa eccede un progetto didattico e si offre di concordare un perimetro su misura.

3.6.3 Tecnologie richieste. Di imposto, in senso stretto, non c'è nulla: R13 raccomanda Flutter o React Native per un unico frontend iOS, Android e web, R9 consiglia il serverless e R14 cita Amazon Bedrock, OpenAI o modelli open source su HuggingFace. Lo stack dell'allegato è però molto preciso: Cognito per identità e MFA, Verified Permissions con policy in Cedar per l'autorizzazione, Lambda, Step Functions ed EventBridge per logica e scadenze, Bedrock con Knowledge Bases e Guardrails, Aurora Serverless v2 con pgvector, S3, KMS e regioni UE, il tutto come Infrastructure as Code. Da non sottovalutare un dettaglio, evidenziato dall'allegato stesso: Comprehend Medical, il NER clinico di AWS, supporta solo l'inglese, quindi sui testi italiani serve un LLM con output strutturato.

3.6.4 Vincoli e requisiti. Il capitolato li divide in quattro gruppi: obbligatori R1–R3 (permessi e ruoli granulari, scadenze con notifiche push ed email, assistente RAG solo su fonti ufficiali verificate), opzionali R4–R6 (le tre evoluzioni appena citate), tecnici e non funzionali R7–R14 (tra cui TLS 1.3 e AES-256, MFA/OAuth2, plugin, WCAG 2.1 AA) e opportunità AI R15–R18. Ci sono poi vincoli impliciti, per nulla secondari: dati sanitari di minori (art. 9 GDPR), quindi residenza in UE, DPIA, consensi revocabili e registro degli accessi; l'assistente non deve mai fare diagnosi né dare indicazioni terapeutiche, pena MDR e AI Act ad alto rischio; nessun dato estratto va scritto senza conferma umana. In attesa di discuterne con la proponente, ipotizziamo un perimetro che sposti il lavoro dall'integrazione alla modellazione, con quattro nuclei legati ai requisiti obbligatori: un'autorizzazione ReBAC verificata, con delega non transitiva e revoca; il Disease Pack come linguaggio di configurazione, con uno schedulatore deterministico delle scadenze; un assistente RAG con citazioni, validità temporale delle fonti e astensione calibrata («non lo so, contatta il tuo Care Navigator»); un registro di audit a prova di manomissione. Fuori R4–R6 e R18, rimandati R16 e R17; da valutare una web app e un monolite modulare in container, ciascuno con il suo ADR.

3.6.5 Aspetti positivi. Il primo, e forse il più sentito, è l'impatto sociale: un problema vero, con utenti veri, che dà un senso concreto ad ogni scelta. Ci ha colpito poi la serietà della proponente, che anticipa da sé i nodi più insidiosi: la cifratura end-to-end in senso stretto è incompatibile con RAG ed estrazione di entità, l'RBAC da solo non basta (il ruolo «insegnante» non dice di quale alunno) e le allucinazioni non si azzerano, si riducono e si misurano con un dataset di valutazione. Oltretutto il perimetro negoziabile lascia spazio a problemi di modellazione veri, da isolare e testare uno alla volta. Il supporto è completo: mentorship tecnica, UX/UI, una referente per il dominio clinico, formazione iniziale e una figura specializzata su richiesta. Ultima ma non meno importante, l'analisi del C6 non va persa: valutare un assistente RAG è parente stretto del valutare una cache semantica.

3.6.6 Criticità e rischi. Il primo nodo è l'ampiezza: presi alla lettera, gli otto contesti di dominio sono soprattutto integrazione di servizi gestiti e nelle nostre circa 150 ore da Programmatore non ci stanno tutti, per cui il perimetro va chiuso prima dell'RTB. L'autorizzazione è il punto più delicato: con deleghe e revoche, stabilire in generale se un diritto potrà mai propagarsi è indecidibile (Harrison, Ruzzo e Ullman), quindi il modello va ristretto apposta per restare analizzabile, e un errore qui espone i dati sanitari di un minore. Fatto sta che molto dipende dalla proponente: corpus di fonti verificate, regole di scadenza (invalidità, PAI/PEI) e dataset di valutazione servono entro l'RTB, altrimenti si costruisce sul vuoto. Mancano criteri di accettazione quantificati (né coverage minima né soglie per l'assistente) e il «design empatico» va reso misurabile. Da non sottovalutare il carico normativo (DPIA, consensi), i costi AWS e dei token non attribuiti e le nostre lacune su AWS, frontend e dominio clinico, che la formazione attenua ma non azzera.

Sintesi del motivo di scelta: Al momento risulta il progetto più completo, potenzialmente stimolante (definito chiaramente il perimetro negoziabile) e meglio organizzato. Soddisfa le aspettative di crescita del team.

3.7 C6: Sistema AI di caching semantico e dashboard FinOps (Var Group)

3.7.1 Descrizione generale. Il capitolato riguarda principalmente lo sviluppo di un sistema di caching semantico e di una dashboard FinOps in estensione a LiteLLM, il proxy open source per il routing multi-provider è già in produzione presso Var Group. L'obiettivo è in sostanza colmarne due limiti critici: la cache riconosce solo query identiche al 100% (quindi si ripagano chiamate per domande semanticamente equivalenti ma testualmente diverse) e manca un riscontro chiaro su costi e consumi, senza aggregazioni per team e progetti né un cruscotto per il consuntivo. Si chiede quindi un layer di caching basato su embedding e similarità (con soglia tarabile, invalidazione per TTL e versionamento, persistenza su Redis o vector DB come Qdrant) che intercetti le query semanticamente simili e ne restituisca la risposta in cache, e un servizio FinOps separato che aggreghi i consumi per team, progetto e utente quasi in tempo reale, calcoli il risparmio reale prodotto dalla cache, gestisca budget e alert e preveda la spesa mensile. L'MVP è un end-to-end funzionante (proxy, cache e dashboard integrati), accompagnato da architettura con ADR, documento dei requisiti post design thinking, diagrammi UML, test automatici con copertura almeno al 70% e repository Git con documentazione e bug report post-collaudo.

3.7.2 Obiettivi e finalità. L'obiettivo principe è ridurre i costi di inferenza verso gli LLM intercettando le query semanticamente sovrapponibili (il capitolato stima che circa l'80% delle query LLM sia semanticamente ridondante) e restituendo risposte dalla cache invece di ricontattare LLM di terzi. Oltre a ciò si chiede visibilità e controllo economico quasi in tempo reale

sui consumi di token, superando lo «scopri il costo a fine fattura». Il tutto estendendo LiteLLM senza reinventare la ruota, con componenti originali, e dentro un contesto, quello Var Group, fatto di sovranità del dato e compliance (GDPR, NIS2, AI Act), con risorse condivise tra tenant ma dati isolati tra team. Le finalità invece sono di far evolvere LiteLLM da semplice proxy di routing ad orchestratore cosciente dei consumi, per ottimizzare il risparmio e rendere visibile dove finisce la spesa; dimostrare pattern architetturali estendibili (non solo con le soluzioni suggerite) e riusabili per funzionalità future del proxy; e, ultima ma non meno importante, la formazione, dato che il progetto richiede molto design thinking, stand-up periodici, il panorama dei modelli open source e un contatto diretto con il mondo del Machine Learning.

3.7.3 Tecnologie richieste. Le tecnologie si dividono in domini interdipendenti, e va detto subito che il capitolato le indica come consigliate: l'unico vincolo vero è LiteLLM come proxy base. Per il proxy lo stack dichiarato è Python 3.10+, LiteLLM (sfruttando le funzioni personalizzate di lettura e scrittura della cache), FastAPI o Flask per gli endpoint e Docker con Docker Compose per ambienti riproducibili. Per gli embedding sono suggeriti text-embedding-3-small di OpenAI o sentence-transformers (open source), con similarità del coseno via numpy/scikit-learn; per il caching invece Redis (cache e TTL), Qdrant o pgvector come vector store e PostgreSQL per le metriche FinOps. Per la dashboard React con Chart.js oppure Streamlit (molto più diretto), su API REST JSON; infine il versionamento su Git, con GitHub o Bitbucket.

3.7.4 Vincoli e requisiti. Architetturalmente LiteLLM resta il proxy base senza reinventarlo, tutta la logica originale sostanziale va su cache semantica e FinOps e il design deve essere modulare ed estendibile; infrastrutturalmente si lavora in multi-tenancy, con dati isolati tra team e progetti. Sul processo sono previsti design thinking, User Story Mapping e una PoC di fattibilità prima dell'implementazione, più stand-up periodici con il referente. Qualitativamente la coverage deve essere almeno al 70% e, cosa fondamentale, i test deterministici vanno separati dalla valutazione probabilistica (soprattutto se più modelli di ML si corroborano a vicenda). Come requisiti funzionali, la cache deve riconoscere query semanticamente simili e non solo identiche, con una soglia tarabile sui dati (forniti e costruiti da noi) e quindi configurabile, invalidare le entry per TTL e versioning e loggare token e latenza risparmiati; la dashboard deve aggregare i consumi per team, progetto e utente, calcolare il risparmio reale prodotto dalla cache, gestire budget con alert automatici in caso di sforamento, prevedere la spesa mensile e aggiornarsi quasi in tempo reale. GDPR, NIS2, AI Act e sovranità del dato vengono dal contesto più che dai vincoli espliciti: se serva un deploy privato su AWS lo chiariremo al design thinking.

3.7.5 Aspetti positivi. Il problema posto è reale per il mercato AI enterprise (costi e GDPR) e stuzzica l'interesse di tutto il gruppo: disegnare una soluzione originale allo spreco di token è senza dubbio complesso, e la complessità è tale da richiedere competenze tecniche interdisciplinari, su un'architettura moderna e molto formativa. Oltretutto lo scope è ben delimitato (anche se con qualche vincolo ambiguo): la proponente non chiede di reinventare LiteLLM, ma di concentrare le energie sui due buchi che contano per loro, isolabili in sottoproblemi testabili ognuno con le proprie metriche. Da non sottovalutare il processo, tra design thinking, PoC, User Story Mapping, implementazione e collaudo, con formazioni dedicate (LiteLLM, embedding e caching, FinOps) e una Q&A ogni due settimane, che formano un canale di comunicazione e feedback strutturato; Var Group, del resto, si è dimostrata molto preparata, con supporto continuo e una pianificazione chiara degli incontri. Infine i criteri di accettazione sono trasparenti (MVP end-to-end, coverage), e questo tutela entrambe le parti da richieste fuori dal dominio del capitolato a metà progetto.

3.7.6 Criticità e rischi. Dopo un paio di iterazioni di analisi e riunioni abbiamo notato delle incoerenze interne, che andranno chiarite al design thinking quando ci verrà assegnato. L'esempio del capitolato («Che tempo fa a Milano oggi?») dipende dal momento, quindi servirlo

dalla cache è un errore e non un risparmio: serve un gate di cacheabilità che stabilisca cosa conviene servire e cosa rigenerare. La premessa «frasi vicine = stesso significato» non regge con la similarità del coseno, perché la vicinanza misura il topic e non l'equivalenza (ne abbiamo discusso anche con la proponente), per cui invece di una soglia configurabile il problema andrebbe riformulato in termini di error rate accettabile. Ci sono poi scenari in cui la cache semplicemente non serve: le interazioni da 50–100k token sono sessioni lunghe, spesso con documenti allegati e dipendenti dal contesto, quindi quasi mai cacheabili, mentre gli hit si concentrano su query brevi single-shot che costano poco; l'80% di ridondanza citato non ha fonti verificabili e il rischio è che la dashboard stessa misuri un risparmio trascurabile. Sono possibili anche leak tra tenant: un hit semantico potrebbe servire una risposta costruita su dati Gold a un utente che non ne ha accesso, e la multi-tenancy compare solo nel FinOps, per cui la cache va partizionata per tenant, livello di sensibilità e permessi. Resta poi la sfida tecnica della concorrenza: più richieste equivalenti in arrivo insieme rischiano di generare tutte un miss e una chiamata al modello, o di scrivere entry in conflitto. Infine LiteLLM si muove in fretta (elenca già un AI Gateway in Rust ancora in beta) e copre in parte i «buchi» con una cache semantica di base e il budget tracking: vanno verificati gli hook Python per tutto il ciclo del progetto e scritto bene il delta rispetto a ciò che esiste.

Sintesi del motivo di scarto: A causa di un errore di formalità, il capitolato è andato perso.

3.8 C7: SmartDashboard

Descrizione: Il progetto SmartDashboard, proposto da Ergon Informatica, mira a sviluppare una piattaforma di Business Intelligence (BI) self-service potenziata dall'Intelligenza Artificiale. Il sistema estrae i dati grezzi da un gestionale ERP, li consolida in un Data Warehouse analitico e offre una web app per la creazione di dashboard interattive tramite un'interfaccia drag & drop. Il fulcro dell'applicativo è l'integrazione di un layer AI basato su LLM che funge da assistente conversazionale: permette di generare visualizzazioni tramite richieste in linguaggio naturale, estrae insight automatici (anomalie, tendenze) e fornisce spiegazioni contestuali sugli indicatori. **Tecnologie:** DuckDB, PostgreSQL/MySQL, Python (Polars/Pandas per l'ETL), React/Angular/Vue, librerie di charting, LLM via API (OpenAI GPT / Anthropic Claude), API REST.

- **Pro:** Architettura modulare a tre livelli ben definita, esplorazione di tecnologie moderne per l'analisi dati (come DuckDB), e un rassicurante approccio di sviluppo incrementale (prima BI, poi AI) delineato dall'azienda.
- **Contro:** Il notevole sforzo richiesto per il Data Engineering (pipeline ETL) potrebbe togliere spazio allo sviluppo dell'app; inoltre, l'uso di LLM commerciali via API limita il lavoro di implementazione IA alla sola integrazione e al prompt engineering.

Sintesi del motivo di scarto: Si riduce a reinventare la ruota creando l'ennesimo strumento di BI ridondante, senza affrontare vere sfide di sistema o di logica.

3.9 C8: Sonar: Documentazione vivente di uno store Shopify

Descrizione: Il progetto mira a realizzare un'applicazione web che si collega a uno store Shopify enterprise, ne interroga API e codice del tema e ricostruisce in automatico la mappa di cosa è installato, di cosa dipende da cosa e di cosa non serve più. I dati acquisiti tramite scansioni durevoli vengono normalizzati in un modello a grafo, salvati come snapshot immutabili e analizzati

da un motore di regole estensibile che segnala orfani e codice morto, indicando fonte e livello di confidenza. Il risultato è una documentazione sempre aggiornata, consultabile via interfaccia web e con diff tra snapshot, esportabile in Markdown/HTML.

Tecnologie: Next.js, Vercel, Supabase, TypeScript, Shopify Admin GraphQL API, Git/GitHub, Drizzle ORM/Prisma, Tailwind CSS, shadcn/ui, React Flow/Cytoscape.js, OpenTelemetry, Vitest, Playwright, MSW, GitHub Actions

- **Pro:** Contesto e-commerce reale e vendibile; output altamente tangibile; stack front-end moderno.
- **Contro:** Vincoli e limiti delle API Shopify; lock-in su piattaforme proprietarie; scarsa flessibilità lato database.

Sintesi del motivo di scarto: Lavorare dentro ecosistemi proprietari chiusi come Shopify impedisce qualsiasi controllo manuale serio sulle fondamenta relazionali e sull'architettura.

3.10 C9: SpecCheck: Skills OpenCode per tracciabilità tra casi d'uso, requisiti e codice sorgente

Descrizione: Il progetto, proposto da Zucchetti, mira a sviluppare SpecCheck, un sistema di estensioni (skill, tool e plugin) per l'agente di harness OpenCode. Per risolvere il problema della verifica nel "vibe coding", la piattaforma automatizza la tracciabilità end-to-end nel ciclo di vita del software: confronta i casi d'uso con le specifiche dei requisiti e verifica che il codice sorgente implementi fedelmente quanto richiesto, segnalando eventuali disallineamenti tramite una dashboard dedicata.

Tecnologie: OpenCode, LLM commerciali via API o open-source, formati di modellazione e documentazione (Mermaid, XMI, Markdown, PDF), Bash/PowerShell, framework per dashboard testuali (TUI) o grafiche (HTML), framework per test automatici.

- **Pro:** Focus su paradigmi IA di ultima generazione ("Vibe Coding"); automazione avanzata del controllo qualità; creazione di tool (Node/JavaScript) e dashboard visive.
- **Contro:** Allontanamento radicale dalla scrittura di codice esplicito tradizionale; analisi delegata a modelli LLM intrinsecamente non deterministici; lock-in sull'agente proprietario OpenCode.

Sintesi del motivo di scarto: Sostituisce l'ingegneria del software classica con la programmazione in linguaggio naturale ("Vibe Coding"), spingendo verso un'astrazione totale gestita dall'IA che azzera il tuo controllo manuale e architetturale sul codice.

4 Conclusioni e Scelta Finale

Al termine del processo di analisi comparativa, il gruppo **Bit Crusher** ha deliberato di presentare formale candidatura per l'assegnazione del seguente progetto didattico:

C5: RareCare

Piattaforma AI per la raccolta dati e il supporto decisionale nelle malattie rare

Proposto da **Miriade srl**

Motivazioni della Scelta:
La scelta non arriva dal nulla: il C5 era già la nostra potenziale terza scelta nella candidatura precedente, rivolta al C6, e l'analisi approfondita ce l'ha confermata. A prima vista RareCare è un'applicazione molto classica, fatta in buona parte di integrazione di servizi gestiti; guardandola da vicino, però, sotto i requisiti obbligatori ci sono quattro problemi di modellazione veri: un'autorizzazione relazionale con delega e revoca da rendere analizzabile, un Disease Pack da trattare come linguaggio di configurazione con scadenze calcolate in modo deterministico, un assistente RAG che deve saper citare le fonti e, quando serve, astenersi, e un registro di audit a prova di manomissione. Problemi che si isolano e si testano uno alla volta, e che il perimetro negoziabile offerto da Miriade ci permette di mettere al centro del lavoro.

Conta poi il resto. L'impatto sociale è concreto, con utenti veri e una validazione sul campo con l'associazione Sanfilippo Fighters; il capitolato è onesto, perché anticipa da sé i propri limiti (cifratura end-to-end, RBAC, allucinazioni) invece di nasconderli; il supporto è completo, dalla mentorship tecnica alla referente per il dominio clinico. Infine la sostenibilità: il preventivo della *Dichiarazione degli impegni* (644 ore, 12.650 €) sposta ore dalla progettazione alla codifica proprio perché l'architettura di riferimento fa già una parte del lavoro, e la consegna è fissata al 16/04/2027, con uno sprint di margine su RTB e PB.

Gestione dei Rischi:
I rischi principali sono tre, e li affrontiamo in ordine di costo dell'errore.

- **Autorizzazione:** un errore espone i dati sanitari di un minore, e un modello troppo espressivo smette di essere analizzabile. Per questo lo restringiamo apposta (niente delega transitiva), scriviamo le policy in Cedar e le sottoponiamo ad analisi automatica, con un proof of concept entro la RTB.
- **Ampiezza:** otto contesti di dominio non ci stanno tutti nelle nostre ore. Il perimetro dell'MVP va concordato con Miriade entro il secondo sprint, lasciando fuori le evoluzioni opzionali.
- **Dipendenza dalla proponente:** corpus di fonti verificate, regole di scadenza e dataset di valutazione servono entro la RTB. Le richieste partono già dal primo sprint, e un ritardo viene dichiarato e ripianificato, non assorbito in silenzio: il silenzio, per noi, non corrobora.

Da non sottovalutare il carico normativo dei dati sanitari: in sviluppo e collaudo useremo solo dati sintetici. Il resto è nella *Dichiarazione degli impegni*.

Il Responsabile di Progetto

